

お祭り (Festival)

Nayra がお祭りで、ラグナ・コロラダへの旅行をかけてゲームをしている。このゲームでは手持ちのトークンを使ってクーポンを購入する。クーポンを購入することで手持ちのトークンが増えることもある。ゲームの目標はできるだけ多くのクーポンを入手することである。

彼女は A トークンを持ってゲームを始める。クーポンは全部で N 枚あり、 0 から $N - 1$ までの番号が付けられている。クーポン i ($0 \leq i < N$) を購入するためには Nayra は $P[i]$ トークンを支払わなければならない、購入の前には最低でも $P[i]$ トークンを持っている必要がある。各クーポンは高々 1 回購入することができる。

さらに、各クーポン i ($0 \leq i < N$) には **タイプ** が割り当てられており、**1 以上 4 以下の整数** $T[i]$ で表される。Nayra がクーポン i を購入すると、彼女の持っている残りのトークン数が $T[i]$ 倍される。形式的には、彼女がゲームのある時点において X トークンを持っており、その状態でクーポン i を購入する場合（このとき $X \geq P[i]$ でなければならない）、購入後の彼女の残りのトークン数は $(X - P[i]) \cdot T[i]$ になる。

あなたの課題は、Nayra が最後に持っている **クーポン** の枚数を最大化するために、彼女がどのクーポンをどの順序で購入するべきかを決めることである。そのような購入手順が複数ある場合、あなたはそれらのうちいずれか 1 つを報告すればよい。

実装の詳細

あなたは以下の関数を実装する必要がある。

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : Nayra がはじめに持っているトークンの数。
- P : それぞれのクーポンの値段を示す長さ N の配列。
- T : それぞれのクーポンのタイプを示す長さ N の配列。
- この関数は各テストケースにおいてちょうど一度だけ呼び出される。

この関数は、Nayra の購入手順を以下のように指定する配列 R を返す必要がある：

- R の長さは、彼女が購入することのできるクーポンの枚数の最大値に一致する必要がある。
- 配列の要素は、彼女が購入するべきクーポンの番号を購入する順に並べたものである。すなわち、彼女は最初にクーポン $R[0]$ を、次にクーポン $R[1]$ を購入し、以降同様である。

- R の要素は互いに異なる必要がある。

購入できるクーポンが 1 つもない場合、 R は空の配列でなければならない。

制約

- $1 \leq N \leq 200\,000$.
- $1 \leq A \leq 10^9$.
- $1 \leq P[i] \leq 10^9$ ($0 \leq i < N$).
- $1 \leq T[i] \leq 4$ ($0 \leq i < N$).

小課題

小課題	得点	追加の制約
1	5	$T[i] = 1$ ($0 \leq i < N$).
2	7	$N \leq 3000$, $T[i] \leq 2$ ($0 \leq i < N$).
3	12	$T[i] \leq 2$ ($0 \leq i < N$).
4	15	$N \leq 70$.
5	27	Nayra は N 枚のクーポンを（ある順序で）すべて購入することができる。
6	16	$(A - P[i]) \cdot T[i] < A$ ($0 \leq i < N$).
7	18	追加の制約はない。

例

例 1

以下の呼び出しを考える。

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra ははじめ $A = 13$ トークンを持っている。彼女は 3 枚のクーポンを以下に示される順序で購入できる：

購入するクーポン	クーポンの値段	クーポンのタイプ	購入後のトークン数
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

この例では、Nayra が 3 枚より多くのクーポンを購入することは不可能であり、上に示されているものが 3 枚のクーポンを購入できる唯一の方法である。したがって、この関数は $[2, 3, 0]$ を返す必要がある。

例 2

以下の呼び出しを考える。

```
max_coupons(9, [6, 5], [2, 3])
```

この例では、Nayra は両方のクーポンをいずれの順序でも購入することが可能である。したがって、この関数は $[0, 1]$ か $[1, 0]$ のいずれかを返す必要がある。

例 3

以下の呼び出しを考える。

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

この例では、Nayra は 1 トークンを持っているが、これはどのクーポンを購入するにも不十分である。したがって、この関数は $[]$ (空の配列) を返す必要がある。

採点プログラムのサンプル

入力形式：

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

出力形式：

```
S
R[0] R[1] ... R[S-1]
```

ここで、 S は `max_coupons` が返した配列 R の長さである。